NAMAA Community at AraSeg Shared Task 2026: Model Diversity, Conditional Decoding, and Missing Punctuation in Arabic Sentence Segmentation

Muhammed Ragab[1,5] **Khloud Al Jallad**[2,5] **Karim Elsayed**[3,5]
Omer Nacar[4,5]

[1]Leibniz Institute for Educational Media Georg-Eckert-Institute [2]Arab International University
[3]American University of Sharjah [4]Tuwaiq Academy
[5]NAMAA Community
[1]muhammed.ahmed@gei.de [2]k-aljallad@aiu.edu.sy
[3]b00092054@aus.edu [4]o.najar@tuwaiq.edu.sa

Abstract

We present NAMAA Community's systems for the four closed-track Arabic sentence-segmentation subtasks of AraSeg 2026. Heterogeneous ensembles combining LoRA-adapted bidirectional language models, XLM-R variants, a character-level SaT model, and punctuation-aware models outperform their development-selected single-model baselines. Large weights assigned to weaker but architecturally distinct members are consistent with complementary errors, although they do not establish causality. A training-only logistic MEMM improves F1 by $+0.9$ and $+0.6$ points on the punctuation-free subtasks but degrades both punctuated ones; the same directional pattern appears on the blind split. Aligning punctuated and punctuation-free inputs reveals $+7.2/+7.1$ F1 points of oracle headroom against the contemporaneous NoPnx locks, while only 52.5% of removed tokens are covered by the restoration label inventory. Genre-proxy analysis localizes the largest degradation to scriptural text. On the blind split, in the closed track, our systems achieved F1 scores of 94.4/91.3/89.9/87.0 on PA, NP, NoPnx-PA, and NoPnx-NP, respectively, ranking 6th/3rd/4th/3rd. The same systems, entered unchanged in the open track, rank 4th/2nd/3rd/3rd. Implementation and experimental resources are publicly available.[1]

1 Introduction

Arabic sentence segmentation is particularly challenging when punctuation is inconsistent or absent (Elkholy et al., 2026a). AraSeg 2026 (Elkholy et al., 2026b) frames it as binary token classification over four aligned versions of the same documents: punctuation and paragraph breaks (**PA**), punctuation only (**NP**), paragraph breaks only (**NoPnx-PA**), and neither cue (**NoPnx-NP**), enabling a controlled study of how each cue affects segmentation. We present NAMAA Community's closed-track systems for all four subtasks.

We investigate three questions. **RQ1**: Does heterogeneous model diversity help beyond standalone models—do character-, subword-, and language-model-based members carry complementary information? **RQ2**: When does sequence decoding help? We evaluate a train-only decoder under both punctuated and punctuation-free conditions. **RQ3**: How much is lost when punctuation is removed, and where? We use oracle map-back, restoration coverage, and genre-proxy diagnostics. The central controlled finding is that the same train-only decoder improves both punctuation-free conditions but harms both punctuated ones, revealing an interaction between decoding and available cues.

2 Related Work

Sentence segmentation is handled either inside a general pipeline, as in Stanza (Qi et al., 2020), or as a dedicated task: FullStop jointly models punctuation and segmentation with Transformer encoders (Vandeghinste and Guhr, 2024), while SaT offers a character-level approach robust to unreliable or absent punctuation (Frohmann et al., 2024). Punctuation restoration is closely related-Cadence casts multilingual recovery as token-level sequence labeling (Pulipaka et al., 2025). These motivate our combination of punctuation-aware, subword-level, and character-level models.

Architecture also matters: bidirectional encoders exploit both left and right context, and decoder-only LLMs can be adapted for sequence labeling by removing causal attention constraints (Dukić and Šnajder, 2024; BehnamGhader et al., 2024). Beyond independent token classification, structured decoding models dependencies between neighbor-

[1]https://github.com/NAMAA-ORG/NAMAA-Community-AraSeg-2026

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 912–921
October 28-29, 2026 ©2026 Association for Computational Linguistics

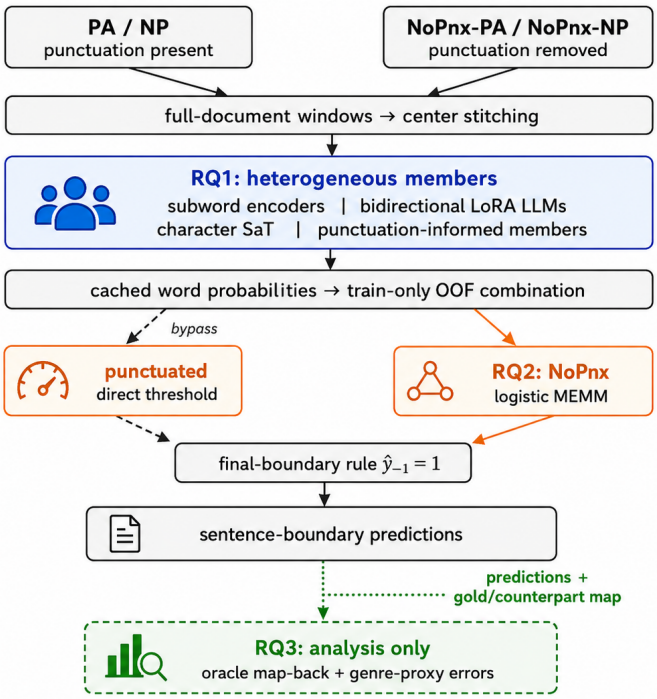


Figure 1: Production inference and post-hoc analysis. Punctuated systems bypass the MEMM; only punctuation-free systems take the amber decoding branch. The green dotted branch uses predictions and gold-aligned counterparts for analysis and is never part of inference. Labels and line styles preserve meaning in grayscale.

ing labels, for which Maximum Entropy Markov Models (MEMMs) are the classical framework (McCallum et al., 2000).

For Arabic, AraSeg provides four aligned conditions that independently vary punctuation and paragraph information (Elkholy et al., 2026a,b) and reports that removing punctuation sharply increases difficulty while larger models do not necessarily dominate lighter encoders in the hardest settings (Elkholy et al., 2026a). We build on that setup by testing heterogeneous ensembles across all four conditions and locating where the loss falls when punctuation disappears.

3 System Overview

Figure 1 summarizes the complete pipeline. All four tracks share full-document windowing, center stitching, and train-only ensemble combination; the punctuated tracks use direct thresholding, whereas the NoPnx tracks additionally evaluate MEMM decoding.

3.1 Document Modeling and Heterogeneous Members

Subword and LLM members classify the first subword of each word, while SaT contributes character-level boundary probabilities aligned to the same word positions. Overlapping windows cover full documents: 512 tokens for encoder models and 2,048 for LLMs, with 128–256 tokens of overlap between consecutive windows (`stride` in the Hugging Face tokenizer API denotes this overlap, not the step size). Center stitching retains each word from the window in which it lies farthest from an edge. Covering the full document instead of truncating at 512 tokens is worth about 8 F1—truncation discarded roughly 54% of true boundaries—but that is a before/after comparison across a bug fix, not an ablation; the overlap itself is worth about 0.3 (Appendix A.5).

The model pool contains Arabic and multilingual encoders (Antoun et al., 2020; Inoue et al., 2021; Abdul-Mageed et al., 2021; Antoun et al., 2021; He et al., 2023; Conneau et al., 2020; Chung et al., 2021), including multi-task and independently seeded XLM-R variants. We also use bidirectional Qwen2.5, Qwen3.5, and Gemma-4 encoders (Qwen Team, 2024, 2026; Gemma Team, 2026), adapted with LoRA (Hu et al., 2022) and R-Drop regularization (Liang et al., 2021). Fine-tuned SaT adds architecturally distinct character-level evidence (Frohmann et al., 2024). For the NoPnx tracks, the candidate pool further includes an XLM-R punctuation auxiliary model and a fine-

tuned Naqta punctuation-restoration encoder (Maroof, 2025). This heterogeneous pool forms the basis of the RQ1 ensemble experiments.

3.2 Train-Only Combination and Conditional Decoding

Member probabilities are cached at document level and combined using either probability/logit averaging or L2-regularized stacked generalization (Wolpert, 1992). Learned combination weights are estimated from five-fold out-of-fold (OOF) predictions over the 174 training documents, so that no training prediction comes from a model that has seen that document. The development split is used only to select the final decision threshold. Candidate combination changes are retained only when paired document bootstrap analysis (Koehn, 2004) yields a 95% confidence interval excluding zero on the held-out practice split.

For RQ2, we additionally evaluate a logistic maximum-entropy Markov model (MEMM) (McCallum et al., 2000). Its features include ensemble member log-odds, document-relative score rank, a local-maximum indicator, document length, a train-derived boundary-density prior, and the gap since the previous predicted boundary. The MEMM is teacher forced during fitting and decoded greedily at inference time. As illustrated in Figure 1, it is applied only to the punctuation-free NoPnx tracks; punctuated tracks bypass the decoder and use direct thresholding.

Finally, every track enforces the document-final boundary constraint $\hat{y}_{-1} = 1$. All 2,632 labeled documents end in a sentence boundary, whereas unconstrained blind-split combiners violated this invariant in 28 of 624 documents. The resulting predictions, with their gold-aligned and punctuated/punctuation-free counterparts, feed the analysis-only RQ3 diagnostics at the bottom of Figure 1.

4 Experimental Setup

Encoders train for 10 epochs at learning rate 1–2×10^{-5} with positive-class weight 2–5. LLM adapters train for 8 epochs at 1–2×10^{-4} with LoRA $r = 16$, $\alpha = 32$, bf16 and R-Drop $\alpha = 0.5$. Locked thresholds for PA/NP/NoPnx-PA/NoPnx-NP are $0.25/0.36/0.46/0.34$. We report the official document-macro P/R/F1 and use 10,000 paired bootstrap resamples for gates. Training uses PyTorch, Hugging Face Transformers/Datasets, PEFT and wtpsplit. Appendix A gives final membership, decoder weights and diagnostic controls; Appendix B is generated from the same results source used for model selection.

5 Results

5.1 Submitted systems and blind evaluation

Table 1 separates the labeled practice test from the blind split. Across the open- and closed-track development leaderboards, we led seven of eight conditions; the exception was open-track NoPnx-PA, where we tied at 87.2 and lost the submission-time tie-break. The blind phase ran from 20 July to 3 August and accepted multiple uploads, so our blind scores changed across dated submissions; every uploaded version had already cleared its prespecified practice gate, so no blind result entered a selection decision. The mean blind change from the practice test is $+0.24$ F1. Per-head changes are not controlled system effects, since the blind corpora differ, but their direction supplies an out-of-sample check.

The blind phase contains 624 documents in total, but only 100–212 for any one subtask, so point estimates must be read with their sample sizes. Even so, two practice-gated re-locks transfer informationally. The NoPnx-NP change that cleared the practice gate at $+0.60$ $[+0.19, +1.01]$ returns $+0.6$ blind ($86.4 \rightarrow 87.0$), all from recall; both figures are scored leaderboard submissions, not estimates. The joint NoPnx-PA gate measures $+0.64$ $[+0.15, +1.16]$ on 262 practice documents but only $+0.1$ on its 100-document blind split. This difference is compatible with sampling noise; it is evidence that the locking protocol did not collapse, not that blind deltas estimate component effects.

5.2 RQ1: Does heterogeneous model diversity help?

Yes, although the evidence is associative rather than causal. Relative to the strongest single-model anchor, the final systems improve by $+1.2/+1.6/+3.2/+4.1$ F1 from PA through NoPnx-NP (Table 2). Architecturally distinct members appear especially useful: adding SaT improves NP and NoPnx-PA by $+0.18$ and $+0.40$ F1 despite weaker standalone NoPnx scores, while Naqta has a lower boundary AUC and low correlation (0.551) with the NoPnx-PA incumbent, yet receives the largest NoPnx decoder coefficient. The strongest controlled check is the NoPnx-PA joint gate: two candidates that were

Subtask / system	P	R	F1	Blind F1	Closed rank	Open rank
PA – logit avg. (3)	95.0	94.6	**94.5**	**94.4**	6/12	4/7
NP – OOF stack (4+SaT)	91.9	94.6	**92.8**	**91.3**	3/10	2/6
NoPnx-PA – MEMM (7+SaT)	87.7	89.3	**87.8**	**89.9**	4/9	3/6
NoPnx-NP – MEMM (6+SaT)	86.1	88.4	**86.5**	**87.0**	3/10	3/7

Table 1: Official practice-test macro P/R/F1 and blind F1, with closed- and open-track ranks. Blind systems include the final-boundary rule. F1 is the document-macro average, so it is not the harmonic mean of the macro P and R shown. Membership and thresholds are in Appendix A.1.

Question	Controlled contrast	PA	NP	NoPnx-PA	NoPnx-NP
RQ1	Final system − dev-selected single-model anchor	$+1.2$	$+1.6$	$+3.2$	$+4.1$
RQ2	MEMM − per-word incumbent, F1 [95% CI]	-0.2 $[-0.5, +0.1]$	-0.1 $[-0.4, +0.1]$	$+0.9$ $[+0.3, +1.6]$	$+0.6$ $[+0.1, +1.0]$
RQ3	Punctuated lock mapped to NoPnx − NoPnx lock	–	–	$+7.2$	$+7.1$

Table 2: The three contrasts that organize the paper. For RQ1, each anchor is the single-model incumbent selected during system development, without using test-set performance for model selection. RQ1 is descriptive because several components change together; RQ2 is the prespecified paired decoder comparison; RQ3 is an oracle bound rather than a deployable system.

individually insufficient jointly improve the original lock by $+0.64$ F1 $[+0.15, +1.16]$, with both precision and recall increasing; the same search on NoPnx-NP is rejected. Overall, the results are consistent with complementary errors from heterogeneous members but do not isolate the causal contribution of any single component.

5.3 RQ2: When does sequence decoding help?

Sequence decoding helps specifically when punctuation is absent. Relative to the per-word incumbent, the MEMM improves NoPnx-PA by $+0.9$ F1 $[+0.3, +1.6]$ and NoPnx-NP by $+0.6$ $[+0.1, +1.0]$, but is rejected on PA and NP (-0.2 and -0.1 F1). The fitted coefficients explain this interaction: with punctuation, local-rank information contributes little, and the decoder mainly learns a document-level boundary-count prior; without punctuation, within-document ranking becomes informative. The same directional pattern appears on the blind split, with both NoPnx scores increasing and both punctuated scores decreasing, although those blind differences are not treated as a second causal estimate. Thus, punctuation presence is a better predictor of decoding utility than nominal reranking headroom.

5.4 RQ3: How much does punctuation removal cost, and where?

Aligned map-back analysis places the cost of punctuation removal at roughly seven F1 points. Applying punctuated PA/NP systems to their aligned NoPnx inputs yields $94.42/92.99$ F1, compared with contemporaneous NoPnx systems at $87.18/85.89$, leaving $+7.2/+7.1$ F1 of oracle headroom ($+6.60/+6.50$ relative to the final submitted systems). Only 52.5% of deleted tokens are covered by the four-label restoration inventory, and this coverage should not be interpreted as recoverable F1.

Restoration helps only when incorporated during training. Frozen Naqta features produce no reliable gain, and restore-then-segment with a frozen punctuated model loses 5.3 F1. By contrast, training on restored text improves NoPnx-PA by $+1.06$ F1 $[+0.25, +1.85]$, while fine-tuning the Naqta encoder improves NoPnx-NP by $+0.60$ $[+0.19, +1.01]$. Genre-proxy analysis further shows that the loss is uneven: Qur'anic verse drops by 16.52 F1 after punctuation removal, whereas Hadith transmission chains lose only $3.48/3.56$ points. These clusters are diagnostic proxies rather than gold genres, so they localize where errors concentrate without establishing a causal genre effect.

6 Conclusion

Heterogeneous members improve Arabic sentence segmentation in every condition, whereas sequence decoding helps only once punctuation is removed: different conditions reward different architectural choices. Oracle analysis puts the cost of that removal at roughly seven F1 points, only part of which restoration models recover, so restoration is for now more useful diagnostically than as preprocessing. Together these support a condition-aware pipeline: heterogeneous scorers on every track, structured decoding only for punctuation-free inputs. The analysis is limited by the modest dev and blind splits, seed variance, proxy genre labels, and controlled deletion in MSA only.

Acknowledgments

We thank Mohammed Mazen Alsayegh for his contributions; the AraSeg 2026 organizers and NAMAA Community for their support; and GWDG Göttingen for KISSKI HPC access. We also thank Tuwaiq Academy for its support.

References

Muhammad Abdul-Mageed, AbdelRahim Elmadany, and El Moatez Billah Nagoudi. 2021. ARBERT & MARBERT: Deep bidirectional transformers for Arabic. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 7088–7105, Online. Association for Computational Linguistics.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2020. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, with a Shared Task on Offensive Language Detection*, pages 9–15, Marseille, France. European Language Resource Association.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2021. AraELECTRA: Pre-training text discriminators for Arabic language understanding. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pages 191–195, Kyiv, Ukraine (Virtual). Association for Computational Linguistics.

Parishad BehnamGhader, Vaibhav Adlakha, Marius Mosbach, Dzmitry Bahdanau, Nicolas Chapados, and Siva Reddy. 2024. LLM2Vec: Large language models are secretly powerful text encoders. In *Proceedings of the First Conference on Language Modeling (COLM)*.

Hyung Won Chung, Thibault Févry, Henry Tsai, Melvin Johnson, and Sebastian Ruder. 2021. Rethinking embedding coupling in pre-trained language models. In *International Conference on Learning Representations (ICLR)*.

Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. 2022. Canine: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91.

Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451, Online. Association for Computational Linguistics.

David Dukić and Jan Šnajder. 2024. Looking right is sometimes right: Investigating the capabilities of decoder-only LLMs for sequence labeling. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 14168–14181, Bangkok, Thailand. Association for Computational Linguistics.

Mohammed Elkholy, Khalid N. Elmadani, Nizar Habash, and Bashar Alhafni. 2026a. Arabic sentence segmentation across genres and punctuation conditions. *Preprint*, arXiv:2606.08025.

Mohammed Elkholy, Khalid N. Elmadani, Nizar Habash, and Bashar Alhafni. 2026b. Arabic sentence segmentation shared task 2026. In *Proceedings of the Fourth Arabic Natural Language Processing Conference*, Budapest, Hungary. Association for Computational Linguistics.

Omar Elshehy, Omer Nacar, Abdelbasset Djamai, Muhammed Ragab, Khloud Al Jallad, and Mona Abdelazim. 2026. AraModernBERT: Transtokenized initialization and long-context encoder modeling for Arabic. In *Proceedings of the Workshop on NLP for Abjad Scripts (AbjadNLP), EACL 2026*.

Fanar Team. 2025. Fanar: An Arabic-centric multimodal generative AI platform. *Preprint*, arXiv:2501.13944.

Markus Frohmann, Igor Sterner, Ivan Vulić, Benjamin Minixhofer, and Markus Schedl. 2024. Segment any text: A universal approach for robust, efficient and adaptable sentence segmentation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11908–11941, Miami, Florida, USA. Association for Computational Linguistics.

Gemma Team. 2026. Gemma 4 technical report. *Preprint*, arXiv:2607.02770.

Pengcheng He, Jianfeng Gao, and Weizhu Chen. 2023. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In *International Conference on Learning Representations (ICLR)*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.

Go Inoue, Bashar Alhafni, Nurpeiis Baimukan, Houda Bouamor, and Nizar Habash. 2021. The interplay of variant, size, and task type in Arabic pre-trained language models. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pages 92–104, Kyiv, Ukraine (Virtual). Association for Computational Linguistics.

Philipp Koehn. 2004. Statistical significance tests for machine translation evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 388–395, Barcelona, Spain. Association for Computational Linguistics.

Xiaobo Liang, Lijun Wu, Juntao Li, Yue Wang, Qi Meng, Tao Qin, Wei Chen, Min Zhang, and Tie-Yan Liu. 2021. R-Drop: Regularized dropout for neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 10890–10905.

Mostafa Maroof. 2025. Naqta: Arabic punctuation restoration with XLM-RoBERTa. Hugging Face model card.

Andrew McCallum, Dayne Freitag, and Fernando C. N. Pereira. 2000. Maximum entropy Markov models for information extraction and segmentation. In *Proceedings of the Seventeenth International Conference on Machine Learning (ICML)*, pages 591–598.

Sidharth Pulipaka, Ashwin Sankar, and Raj Dabre. 2025. Mark my words: A robust multilingual model for punctuation in text and speech transcripts. In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pages 1758–1776, Mumbai, India. The Asian Federation of Natural Language Processing and The Association for Computational Linguistics.

Peng Qi, Yuhao Zhang, Yuhui Zhang, Jason Bolton, and Christopher D. Manning. 2020. Stanza: A python natural language processing toolkit for many human languages. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 101–108, Online. Association for Computational Linguistics.

Qwen Team. 2024. Qwen2.5 technical report. *Preprint*, arXiv:2412.15115.

Qwen Team. 2026. Qwen3.5: Towards native multimodal agents. Blog post.

Vincent Vandeghinste and Oliver Guhr. 2024. FullStop: Punctuation and segmentation prediction for Dutch with transformers. *Language Resources and Evaluation*, 58:1335–1354.

David H. Wolpert. 1992. Stacked generalization. *Neural Networks*, 5(2):241–259.

A Reproducibility and Diagnostic Details

A.1 Final systems and training configuration

Class weight is 2–5 by subtask. Encoder/LLM windows are 512/2,048 tokens with 128–256 tokens of overlap. Encoders train for 10 epochs at LR 1–2×10^{-5}; LLMs train for 8 epochs at 1–2×10^{-4} with LoRA $r = 16$, $\alpha = 32$, bf16, and R-Drop $\alpha = 0.5$. OOF uses five folds and the seed 42. The final NoPnx-PA membership came from a joint 15-subset gate: two individually rejected candidates jointly improved the original lock by $+0.64$ F1 $[+0.15, +1.16]$; applying the same procedure to NoPnx-NP was rejected.

A.2 Decoder features and fitted weights

The MEMM uses member log-odds, mean score, local maximum/rank, relative position, log document length, train boundary density, log gap since the last boundary and a short-gap indicator. The two most diagnostic coefficients are:

Subtask	Boundary density	Local rank
PA	-4.53	-0.19
NP	-5.13	$+0.05$
NoPnx-PA	-1.65	$+0.70$
NoPnx-NP	-1.07	$+0.37$

A.3 Punctuation-restoration ladder

Naqta is `MostafaMaroof/Naqta`, a public XLM-R-large Arabic punctuation-restoration model. Frozen, its terminal-probability feature changes NoPnx-PA/NoPnx-NP by $+0.26$ $[-0.08, +0.63]$ and $+0.15$ $[-0.26, +0.54]$; both are noise. Inserting marks and applying a frozen punctuated lock loses 5.3 F1. Training on restored text changes the two heads by $+1.06$ $[+0.25, +1.85]$ and $+0.13$ $[-0.96, +1.08]$; fine-tuning Naqta's encoder changes them by $+0.41$ $[-0.07, +0.88]$ and $+0.60$ $[+0.19, +1.01]$. Thus, each trained-in route is significant on a different head; the robust contrast is a borrowed signal versus a signal incorporated during training, not a general claim that restoration works.

A.4 Genre-Proxy Analysis of Punctuation Removal

To examine whether the effect of punctuation removal varies across document types, we perform an exploratory genre-proxy analysis on the practice-test documents. Because gold genre labels are unavailable, documents are represented with character-n-gram TF-IDF features and partitioned into five clusters using KMeans. We then inspect each cluster and assign a descriptive label based on its dominant textual characteristics. These labels are therefore *analyst-assigned proxies*, not corpus annotations. The analysis is intended to localize where punctuation removal causes the largest degradation, rather than to establish genre-level performance estimates.

The degradation after punctuation removal is strongly cluster-dependent. Qur'anic verse shows the largest PA→NoPnx-PA loss (-16.52 F1), followed by numbered scripture/scripts (-11.31) and enumerated short-unit text (-7.66). The same broad pattern appears for NP→NoPnx-NP, where Qur'anic verse loses 12.53 points and enumerated

Subtask	Members and combiner	Thr.
PA	`e25 e32 e33`; logit average	0.25
NP	`e40 e15 e16 e38` + fine-tuned SaT; OOF stack	0.36
NoPnx-PA	`e41 e17 e18 e11 e38 e69 e76` + SaT; MEMM	0.46
NoPnx-NP	`e42 e19 e20 e12 e38 e70` + SaT; MEMM	0.34

Table 3: Final submitted systems. All learned combination/decoder weights use five-fold OOF train predictions; dev selects the listed threshold.

			PA			NP			NoPnx-PA†			NoPnx-NP†		
Genre-proxy cluster	n	**Mean seg.**	P	R	F1	P	R	F1	P	R	F1	P	R	F1
Modern prose	121	14.5	93.61	93.25	93.10	89.46	93.74	91.16	86.96	90.84	88.55	85.18	89.02	86.75
Numbered scripture/scripts	58	13.1	95.21	93.75	94.15	90.01	92.37	90.51	82.30	85.43	82.84	79.73	84.72	81.18
Enumerated short-unit text	45	7.2	99.12	99.73	99.40	99.14	99.88	99.49	94.02	91.02	91.74	93.10	87.73	89.49
Hadith transmission chains	20	10.2	89.19	88.67	87.94	88.67	90.01	88.37	84.51	86.59	84.46	82.95	88.65	84.81
Qur'anic verse	18	19.0	100.00	100.00	100.00	100.00	100.00	100.00	87.42	82.99	83.48	88.04	88.05	87.47
Overall	262	–	95.01	94.59	94.49	91.91	94.64	92.84	86.99	88.81	87.18	85.36	87.75	85.89

Table 4: Document-macro precision, recall, and F1 across five genre-proxy clusters on the practice test. **Mean seg.** denotes mean gold segment length. Clusters are derived from character-n-gram TF-IDF representations with KMeans ($k = 5$); cluster descriptions are analyst assigned and should not be interpreted as gold genre labels.

short-unit text loses 10.00. By contrast, Hadith transmission chains show the smallest reduction in both contrasts (-3.48 and -3.56), with modern prose also relatively stable (-4.55 and -4.41). These results suggest that punctuation removal is most damaging in clusters whose observed boundaries were strongly exposed by surface formatting or punctuation cues. Given the small size of several clusters and the absence of gold genre labels, we treat this analysis as diagnostic rather than as a genre-level benchmark.

For reproducibility, the NoPnx columns marked with † use the 2026-07-13 decoder locks (87.18/85.89 overall), because the later practice-test prediction files for the submitted systems were unavailable. PA and NP use the submitted locks. Consequently, the cluster analysis should not be used to recompute the final submitted NoPnx scores.

A.5 Cross-Family Encoder Sweep and Window Overlap

The main experiment ledger in Table 6 records our numbered campaign, but it does not include a separate cross-family encoder sweep conducted under a common training recipe. That sweep evaluates six encoders using 15 epochs, learning rate 2×10^{-4}, batch size 4 with gradient accumulation of 2, warmup ratio 0.1, and weight decay 0.05, with the decision threshold tuned on development data. It adds two architecture families not represented in the numbered campaign—multilingual BERT and the tokenizer-free Canine-s model (Clark et al., 2022)—as well as a stronger CAMeL-BERT-MSA

Model	**PA**	**NP**	**NoPnx**	
			PA	**NP**
mBERT-cased	87.87	–	68.65	–
mBERT-uncased	87.95	–	68.55	–
CAMeL-BERT-Mix	90.32	–	78.12	–
CAMeL-BERT-MSA	90.81	–	79.55	–
RemBERT	92.55	90.57	83.68	80.10
Canine-s (char.)	86.78	79.51	60.61	45.76

Table 5: Test macro F1 for a cross-family encoder sweep under a shared training recipe.

rerun. Because these are single runs without paired bootstrap analysis, we use them only as supporting diagnostics and do not use them to re-lock any submitted system.

The most informative comparison is the behavior of Canine-s under punctuation removal. Its PA score is 86.78 F1, but it falls by 26.17 points on PA→NoPnx-PA. This drop is substantially larger than those of mBERT-uncased (19.40), mBERT-cased (19.22), CAMeL-BERT-Mix (12.20), CAMeL-BERT-MSA (11.26), and RemBERT (8.87). Among the two models evaluated on all four subtasks, Canine-s falls 41.02 points from PA to NoPnx-NP, compared with 12.45 for RemBERT; correspondingly, the RemBERT-Canine-s gap widens from 5.77 F1 on PA to 34.34 on NoPnx-NP.

This pattern is consistent with the RQ3 finding that removing punctuation changes which model properties are useful, but it should not be interpreted as independent evidence for a character-level effect. Canine-s is the only tokenizer-free model

in this sweep, is represented by a single run, and has no confidence interval; tokenizer design and character granularity are, therefore, confounded.

The same sweep also shows a common tendency toward over-segmentation. Against a gold average of 47.4 boundaries per document, all six models predict more boundaries, with excess counts ranging from +1.1 for RemBERT to +4.5 for Canine-s. This agrees with the precision-below-recall pattern observed for three of the four submitted systems in Table 1.

Window overlap. We additionally examine whether performance depends on the amount of overlap between adjacent inference windows. Here, *overlap* denotes the number of tokens shared by consecutive windows, rather than the window step size. The parallel sweep suggested an improvement of roughly +0.2 F1 from using 128-token overlap. We test this more directly by holding checkpoint `e25` (XLM-R-Large, trained with 256-token overlap) fixed and varying only the inference-time overlap, with each setting evaluated at its own development-tuned threshold.

Relative to the 256-token training setting, zero overlap loses −0.29 F1 with a 95% interval of $[-0.44, -0.15]$. Overlaps of 64, 128, and 384 tokens yield differences of −0.10 $[-0.26, +0.06]$, −0.03 $[-0.18, +0.13]$, and −0.02 $[-0.16, +0.10]$, respectively. Thus, the evidence supports the importance of having some overlap, while providing little evidence that the precise overlap size matters within the 64–384 token range. It also suggests that moderate train-inference overlap mismatch is not detectably harmful in this range. This controlled result is consistent with the approximately +0.2 effect observed in the parallel sweep and indicates that the 128–256 token range used in Section 3 should be viewed as a robust operating range rather than a finely tuned hyperparameter.

B Full Experiment Ledger

The ledger summarizes the single-model runs with usable recorded results. Multi-task runs are reported separately for each evaluated subtask, including the NP result for `e38`. Fine-tuned SaT is also included for all four subtasks; development F1 was not recorded for these runs. Early `e1`/`e2` recipes are marked with a dagger because they precede the tuned reruns, while `e80`–`e85` are second-seed replications rather than distinct systems. A dash denotes an unavailable value, not zero. Ensembles, decoders, gating experiments, and oracle analyses are excluded because they are not standalone model runs. We additionally cite AraModernBERT-Base V1.0 (Elshehy et al., 2026) and Fanar-1-9B-Instruct (Fanar Team, 2025), which appear only in the ledger and are not discussed in the main analysis.

Table 6: Full experiment ledger. Test F1 uses the official document-macro scorer; dev F1 is reported at the selected threshold.

Exp.	Model	Subtask	Key technique	Test F1	Dev F1
e1†	AraBERT-large-v02	PA	Plain fine-tune	84.57	89.40
e2†	CAMeL-BERT MSA	PA	Plain fine-tune	87.15	88.57
e3	XLM-R-Large	PA	Plain fine-tune	89.82	90.46
e6	AraModernBERT-Base	PA	8192-token whole-document context	79.04	80.98
e7	XLM-R-Large	NoPnx-PA	`pos_weight`= 3	–	–
e8	XLM-R-Large	NP	`pos_weight`= 3	90.05	90.22
e9	XLM-R-Large	NoPnx-NP	`pos_weight`= 5	–	76.22
e10	XLM-R-Large	PA	`pos_weight`= 3	90.06	90.35
e11	XLM-R-Large	NoPnx-PA	`pos_weight`= 2	81.64	81.65
e12	XLM-R-Large	NoPnx-NP	`pos_weight`= 2	81.52	81.76
e13	XLM-R-Large	PA	Seed 1	89.80	89.90
e14	XLM-R-Large	PA	Seed 2	90.50	90.68
e15	XLM-R-Large	NP	Seed 1	90.51	91.17
e16	XLM-R-Large	NP	Seed 2	89.88	90.96
e17	XLM-R-Large	NoPnx-PA	Seed 1	81.47	81.89
e18	XLM-R-Large	NoPnx-PA	Seed 2	81.91	82.03
e19	XLM-R-Large	NoPnx-NP	Seed 1	82.11	82.31
e20	XLM-R-Large	NoPnx-NP	Seed 2	81.78	81.76
e21	mDeBERTa-v3-base	PA	Multilingual encoder	90.77	91.67
e22	MARBERTv2	PA	Arabic encoder	90.53	91.09
e23	AraELECTRA-base	PA	ELECTRA discriminator	91.03	91.76
e24	XLM-R-XL (3.5B)	PA	Increased model capacity	92.36	93.01
e25	XLM-R-Large	PA	Window stride 256	91.24	91.86
e26	XLM-R-Large	PA	BiLSTM head	91.31	91.78
e27	XLM-R-Large	PA	Focal loss ($\gamma = 2$)	90.31	90.93
e28	XLM-R-Large	PA	FGM adversarial training	90.89	91.03
e29	XLM-R-Large	PA	SWA, epochs 9–12	90.12	90.04
e30	XLM-R-Large	PA	BiLSTM + FGM + SWA	91.42	91.78
e31	Qwen2.5-7B-Instruct	PA	LoRA	88.80	90.87
e32	Qwen3.5-9B	PA	LoRA	93.34	94.07
e33	Gemma-4-12B	PA	LoRA	91.87	94.15
e34	Qwen3.5-9B	PA	LoRA + R-Drop	93.22	93.82
e35	Qwen3.5-9B	PA	LoRA + LLRD	92.24	92.78
e36	Qwen3.5-9B	PA	LoRA + R-Drop + LLRD	92.18	92.88
e37	XLM-R-Large	PA	CRF head (Viterbi)	91.36	91.64
e38	XLM-R-Large	NP	Multi-task, all four tracks	91.01	–
e38	XLM-R-Large	NoPnx-NP	Multi-task, all four tracks	81.97	82.16
e39	XLM-R-Large	NoPnx-PA	Distillation from e32	81.40	82.22
e40	Qwen3.5-9B	NP	LoRA + R-Drop	90.67	91.36
e41	Qwen3.5-9B	NoPnx-PA	LoRA + R-Drop	84.59	86.03
e42	Qwen3.5-9B	NoPnx-NP	LoRA + R-Drop	81.79	82.37
e48	XLM-R-Large	NoPnx-NP	Distillation from e32	81.67	82.44
e50	XLM-R-Large	NoPnx-NP	Recall loss, seed 42	78.81	75.18
e51	XLM-R-Large	NoPnx-PA	Recall loss, seed 42	78.58	74.36

Exp.	Model	Subtask	Key technique	Test F1	Dev F1
e52	XLM-R-Large	NoPnx-NP	Recall loss, seed 2	78.42	73.29
e53	XLM-R-Large	NoPnx-PA	Recall loss, seed 2	76.80	71.84
e54	XLM-R-Large	NP	Recall loss, seed 42	89.09	86.73
e55	XLM-R-Large	PA	Recall loss, seed 42	89.74	87.89
e56	XLM-R-Large	NoPnx-NP	Frozen syntax features	77.34	73.10
e57	XLM-R-Large	NoPnx-PA	Frozen syntax features	78.25	74.56
e58	XLM-R-Large	NP	Frozen syntax features	88.52	83.87
e59	XLM-R-Large	PA	Frozen syntax features	89.29	88.49
e60	XLM-R-Large	NoPnx-NP	Auxiliary head, $p = 1.0$	79.45	74.76
e61	XLM-R-Large	NoPnx-PA	Auxiliary head, $p = 1.0$	79.59	75.72
e62	XLM-R-Large	NoPnx-NP	Auxiliary head, $p = 0.5$	76.71	71.79
e63	XLM-R-Large	NoPnx-PA	Auxiliary head, $p = 0.5$	76.77	72.40
e64	AraBERT-large-v02	PA	Tuned rerun of e1	91.09	92.05
e65	Fanar-1-9B-Instruct	PA	LoRA, e32 recipe	93.24	93.50
e66	RemBERT	PA	Tuned rerun	93.36	94.08
e69	Naqta encoder	NoPnx-PA	Punctuation-restoration initialization	82.49	83.01
e70	Naqta encoder	NoPnx-NP	Punctuation-restoration initialization	82.42	82.95
e71	XLM-R-Large	NoPnx-PA	Trained on Naqta-restored text	78.34	82.35
e72	XLM-R-Large	NoPnx-NP	Trained on Naqta-restored text	77.76	82.33
e73	Qwen3.5-27B	PA	LoRA, bf16	93.92	94.74
e74	Qwen2.5-7B	PA	LoRA	91.73	92.65
e75	XLM-R-Large	NoPnx-PA	Comma-restored retraining	76.77	82.89
e76	XLM-R-Large	NoPnx-PA	Comma-restored retraining	78.84	83.06
e80	Qwen3.5-9B	PA	e32 recipe, seed 2	93.75	93.93
e81	Qwen3.5-9B	NP	e40 recipe, seed 2	90.91	91.39
e82	Qwen3.5-9B	NoPnx-PA	e41 recipe, seed 2	84.63	85.78
e83	Qwen3.5-9B	NoPnx-NP	e42 recipe, seed 2	81.69	82.01
e84	Gemma-4-12B	PA	e33 recipe, seed 2	92.22	93.23
e85	Naqta encoder	NoPnx-NP	e70 recipe, seed 2	83.11	83.24
e87	XLM-R-Large	NoPnx-PA	Comma-restored retraining	77.53	82.79
SaT	Fine-tuned SaT	PA	Character-level segmenter	91.19	–
SaT	Fine-tuned SaT	NP	Character-level segmenter	91.21	–
SaT	Fine-tuned SaT	NoPnx-PA	Character-level segmenter	79.54	–
SaT	Fine-tuned SaT	NoPnx-NP	Character-level segmenter	79.55	–